QueryAPIGate DOCUMENTATION · REFERENCE

MCP server

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=MCP

`queryapigate mcp` exposes your saved queries as MCP tools, so an AI agent can list and call them directly - the same way `/catalog` / `/openapi.json` already let a human or a REST client discover the same thing - instead of shelling out to `curl` against the REST API. Two more tools, not tied to any saved query, let an agent work with a connection directly: `list_tables` (schema discovery) and `execute_sql` (read-only ad-hoc SQL) - see Ad-hoc tools below.

What's exposed

Read-only saved queries (`SELECT` / `WITH` / `SHOW` / `DESCRIBE` / `EXPLAIN`, and any Mongo `find()` query - the same classification the REST API's own write guard already uses) become MCP tools. A saved query that writes becomes one only when its author marked it for agents, for a key whose grant allows the write - and then a person approves each write first (write tools); otherwise it isn't listed, and isn't run when called by name. Ad-hoc SQL never writes: `execute_sql` (below) always runs read-only over MCP, regardless of what the calling key would otherwise be allowed to do over REST.

Tool listing (`tools/list`) honors the exact same per-key scoping `GET /catalog` does - `connections` / `queries` / `collections` grants, `apikeys.can_run_saved()` - so an agent's key never sees or calls more than the identical REST key could. A tool's `inputSchema` is generated from the saved query's own declared parameters, the same JSON Schema translation `/openapi.json` already uses.

Calling a tool (`tools/call`) runs the query through the exact same code path `GET /q/<name>` does: the same connection-grant check, parameter validation, `cache_ttl` caching, `execution_history` recording and audit trail. A run triggered over MCP shows up in the admin UI's History tab and the Home tab's live feed exactly like a REST call would - there is no separate, parallel execution path to keep in sync.

Structured results (`outputSchema`)

Every tool - a saved query, `list_tables`, or `execute_sql` - declares an `outputSchema`, and every successful call returns a matching `structuredContent` alongside the usual text content (the text is a serialized copy of the same object, kept for clients that don't read `structuredContent`). A row-returning tool (a saved query or `execute_sql`) always returns `{"rows": [...], "truncated": <bool>}`; `list_tables` returns `{"tables": [...], "truncated": <bool>}`. This is deliberately a generic envelope, not a per-column schema - a saved query's actual result columns are only known once it runs, not from its stored definition, so declaring real per-column types would need a new admin-authored field this version doesn't add.

Breaking, if you already have an MCP integration: before this, a row-returning tool's text content was a bare JSON array of rows with an ad-hoc `"(truncated to N rows)"` suffix string when capped. It is now always the wrapped `{"rows": [...], "truncated": <bool>}` object - read `structuredContent` directly rather than parsing the text block, or parse the text block as this new object shape instead of a bare array.

Ad-hoc tools: `list_tables` and `execute_sql`

Not every question an agent needs to answer has a saved query behind it yet. Two fixed tools - not tied to any saved query - are listed for a key whenever it has any connection-level access at all (the same `connections` grant `/execute_sql` and `/connections/<name>/schema` already check over REST; a key scoped only to specific queries or collections never sees these two, the same way it can't reach those REST endpoints either):

- **`list_tables`** - `{"connection_name": "...", "database": "..."}` (`database` is admin-key only, same as REST). Returns the same shape `GET /connections/<name>/schema` does: each table/view (or Mongo collection) with its columns, types, nullability, and primary/foreign keys where the dialect supports it.
- **`describe_table`** - `{"connection_name": "...", "table": "...", "database": "..."}`. One table's columns, types and primary/foreign keys - the same, masking-aware, so a masked or dropped column is flagged rather than shown as readable. A table the key can't reach answers exactly as a missing one (`table_not_found`), so it can't be used to probe for a table outside the key's `allowed_tables`. Handy before writing an `execute_sql`.
- **`execute_sql`** - `{"connection_name": "...", "sql": "...", "params": {...}, "page_size": N, "database": "..."}`. Runs a single read-only statement - **always read-only over MCP**, regardless of the calling key's own `allow_writes` grant; a key's `allowed_tables` restriction, if it has one, is still enforced. Results are capped to `QUERYAPIGATE_MCP_MAX_ROWS` the same way a saved-query tool call is.

Both reuse the exact permission checks and execution code the REST API already has - there is no separate ad-hoc-SQL path for MCP to drift out of sync with REST's own. An `execute_sql` call is recorded in run history like a `POST /execute_sql` one, with `"transport": "mcp"` - see Ad-hoc runs.

Running it

```shell
pip install "queryapigate[mcp]"
queryapigate mcp
```

It listens on its own port (`QUERYAPIGATE_MCP_PORT`, default `5001`) - separate from `queryapigate serve`'s port, so both can run side by side against the same `QUERYAPIGATE_HOME`. The MCP endpoint is `http://<host>:<port>/mcp`, speaking Streamable HTTP; `/metrics` and `/health` on the same port report on this process.

It's a **separate process** from the REST server, not a mode of it: MCP's HTTP transport is ASGI-native, while QueryAPIGate's REST API is WSGI (Flask on gunicorn). What actually matters for correctness - reusing this project's own connection-grant checks, caching and audit logging by direct Python import, not an HTTP round-trip back to the REST API - holds regardless of which process each server runs in.

Authentication

Same as the REST API: an `X-API-Key` header, checked against the same `QUERYAPIGATE_API_KEY`/scoped keys - or, when signed-in users are enabled, an `Authorization: Bearer` token, so an agent acting for a user gets that user's role and its `from_claim` row filters. A call carrying an API key is judged on that key alone; a wrong key never falls back to the token. Send either as a request header, the way any HTTP-capable MCP client configuration allows - never as a URL parameter, which would leak it into logs. An unset server key means open access, same as the REST API: local (loopback) clients only, and any other is refused with a `403 open_access_remote` before its session starts. That includes a client reaching `docker run -p`, which arrives from the container's network, not loopback: in Docker, set `QUERYAPIGATE_API_KEY`. Before 0.47 the MCP server let any client in, as an unrestricted admin.

Every request to `/mcp` is checked before the MCP session sees it: a missing or wrong credential is an HTTP `401` with a `WWW-Authenticate` header (until 0.17 it was a tool error inside a `200`), a refused one a `403`, and `QUERYAPIGATE_RATE_LIMIT` a `429` - each with the `{"error", "code", "request_id"}` body REST uses.

Sign-in for AI clients (OAuth)

> **Stable** since 0.30, under the compatibility promise (what that means). Settings, and scopes and claims it reads, may still be added as the MCP authorization specification grows; the answers to a refused token stay as they are. Limits: QueryAPIGate checks tokens and issues none, so registering clients and revoking tokens are the provider's; a token must be a JWT signed with an asymmetric key - an opaque one, which only introspection could check, is refused; and a token is valid until it expires.

A hosted AI client - a connector in an assistant - doesn't take an API key: it follows the MCP specification's authorization, OAuth 2.1. The MCP server takes the part the specification gives it, a **resource server**: it checks tokens, and your **identity provider** (Keycloak, Okta, Auth0, Entra ID, ...) issues them, signs the user in and registers the client. QueryAPIGate never sees a password.

VARIABLE	DEFAULT	EFFECT
`QUERYAPIGATE_MCP_RESOURCE_URL`	unset	The MCP server's URL as clients reach it, e.g. `https://data.example.com/mcp`. Every token must name it as its audience.
`QUERYAPIGATE_MCP_AUTH_SERVER`	unset	The identity provider's issuer URL, e.g. `https://idp.example.com/realms/acme`. Its keys are found through its metadata (RFC 8414 or OpenID Connect discovery).
`QUERYAPIGATE_MCP_SCOPE`	`queryapigate.mcp`	The scope a token must carry.
`QUERYAPIGATE_MCP_ROLE`	unset	The role every agent signed in this way gets...
`QUERYAPIGATE_MCP_ROLE_CLAIM`	unset	...or the token claim naming its role (a string or a list; the first existing role wins).

Set the first two, and a role, to turn it on; anything half-set, or not `https://` (except `localhost`), stops startup. Then:

- **Discovery.** `GET /.well-known/oauth-protected-resource` (and `.../mcp`) is the server's Protected Resource Metadata (RFC 9728): its URL, `authorization_servers`, `scopes_supported`. A `401` points at it: `WWW-Authenticate: Bearer resource_metadata="https://data.example.com/.well-known/oauth-protected-resource/mcp", scope="queryapigate.mcp"` - which is how a client knows where to sign the user in.
- **What a token must be:** signed by the provider's published keys, with the key's own algorithm; `iss` the provider; `aud` the MCP server's URL; not expired; carrying the scope (as `scope`, or `scp`; for a scope named by URI, `api://queryapigate/queryapigate.mcp`, its last part). Anything else is `401` (`error="invalid_token"`). A valid token without the scope is `403 insufficient_scope`; one whose user has no role here, or whose role's `allowed_ips` leave out the address, is `403 forbidden` - signing in again wouldn't help.
- **Tokens are kept apart.** A token issued for the REST API isn't accepted on `/mcp`, however valid, and REST refuses a token naming the MCP server - so one leaked from either can't be replayed against the other. `QUERYAPIGATE_JWT_AUDIENCE` can't include the MCP server's URL. With OAuth on, a bearer token on `/mcp` must be one issued for it; API keys still work, for SDKs and scripts. No token is ever passed on: databases are reached with the connection's own credentials.

- **What the agent may do** is its role's: connections, saved queries, `allowed_tables`, its rate limit, and its masking rules. `from_claim` parameters bind to the token's claims, so an agent acting for a person sees only that person's rows. Its runs are in history as `jwt:<sub>`, with `"transport": "mcp"` - the same name that person has signed in to REST, when REST trusts the same issuer (or signs nobody in). When REST trusts another issuer, whose users' `sub`s could be the same strings for other people, the agent is `mcp:<sub>` instead, so two people never share one name, rate limit or event stream.
- **Registering the client** is between the client and your provider: Client ID Metadata Documents or Dynamic Client Registration where the provider supports them, or an administrator registering it by hand. QueryAPIGate plays no part, and issues no tokens of its own.

The step by step, with Keycloak: let AI clients sign in through your identity provider.

Descriptions and the data dictionary

> **Stable** since 0.30, with descriptions themselves - what may still be added, and the limits, are there.

What an agent is told the data means:

- **`list_tables`** carries each table's and column's `description`, the connection's `notes`, and `masked` on each column the caller gets masked.
- **A saved-query tool's output schema** describes its result columns, from the query's `columns`.
- **The data dictionary** is an MCP resource per connection the caller may use - `queryapigate://connections/<name>/dictionary`, Markdown: the notes, then every table it can reach, each column with its type, its description and whether it's masked for this caller. `resources/list` lists them; `resources/read` returns one. Callers with saved queries only (no `connections` grant) have none.

The step by step: tell AI agents what your data means, and see what they did.

Write tools: agents that write, approved by a person

> **Stable** since 0.30, under the compatibility promise (what that means). Fields of a write request, and modes, may still be added; `approval` and `idempotency_key` stay as they are. Limits: one `INSERT`, `UPDATE` or `DELETE` per tool, on PostgreSQL, MySQL (transactional tables), SQLite or DuckDB; and an approver sees the SQL and the arguments, not which rows will change.

Agents read only - except through a saved query whose author marked it **`agent_writes`**, for a caller whose grant allows that write (`{"name": "close_ticket", "allow_writes": true}`, as over REST), with `QUERYAPIGATE_ALLOW_WRITES` on, on PostgreSQL, MySQL, SQLite or DuckDB. Such a query is one `INSERT`, `UPDATE` or `DELETE` - never DDL - and is listed as a tool with `readOnlyHint: false` and `destructiveHint: true`. `execute_sql` never writes.

- **`"agent_writes": "approval"`** - the first call writes nothing: it answers `{"status": "approval_required", "request": "wr_...", "approve_at": ".../console/approvals/wr_..."}`. An administrator whose role has `writes.approve` (owners, admins) approves or rejects it - `POST /api/v1/write-requests/{id}/approve` or `/reject` with a `reason`. The agent then calls again with the same arguments and `approval: "wr_..."`: it runs if approved, for this caller, with exactly these arguments, before it expires (`QUERYAPIGATE_WRITE_APPROVAL_MINUTES`, default

15. ▪ and once: another call with the same approval returns the first one's result.

- **`"agent_writes": "insert"`** - a single `INSERT` runs at once, with an optional `idempotency_key` so a retry within a day returns the first call's result instead of inserting again.
- **`max_affected_rows`** (default 1) - the write runs in a transaction, rolled back when it changed more rows (`too_many_rows_affected`, nothing changed). On MySQL, keep these queries on InnoDB tables: a MyISAM table can't be rolled back.
- What it returns (`RETURNING`) is masked like any read. Every request is kept 30 days (`GET /api/v1/write-requests`); the run is in history with `request` and `approved_by`; approvals and rejections are in the audit log.
- `QUERYAPIGATE_CONSOLE_URL` (`https://data.example.com`) makes `approve_at` a full link; without it, it's the Console path.
- **In the Console: Observability → Approvals**, with a count of the writes waiting in the sidebar; the link an agent is given opens its request there - the agent, the SQL, the exact arguments, the row ceiling, when it expires - with **Approve** and **Reject** (and a reason). The `writes_awaiting_approval` alert says when writes are waiting, and the Agents screen counts each agent's write requests by how they ended.
- **Four eyes:** a write is never approved by the credential that asked for it (`own_write`) - give agents their own keys. An agent with 100 writes waiting is refused more (`too_many_pending_writes`).

The step by step: let an AI agent write - with a person approving each write.

Governance: the same rules as REST

Every tool call passes through the same checks a REST request does, in the same order, by calling the same code (`queryapigate/governance.py`):

1. **`QUERYAPIGATE_RATE_LIMIT`**, per client address, before authentication - so guessing keys is throttled too.
2. **Authentication**, as above.
3. **The caller's own `rate_limit` grant** (a key's, or a signed-in user's role's).
4. **Grants** - connections, queries, collections, `allowed_tables`, IP allowlists - then the tool itself.

A refused call (past authentication, which is HTTP-level - above) is a tool error (`isError: true`) whose text says why, and whose `structuredContent` carries the same `{"error", "code"}` a REST error body does - `rate_limited` (with `retry_after`), `unauthorized`, `table_not_allowed`, `read_only`, `param_invalid` and the rest listed under Errors - so an agent can branch on the code instead of parsing the message. Every call is counted in `/metrics`' request counters and latency histograms with method `MCP` and endpoint `mcp.saved_query`, `mcp.execute_sql` or `mcp.list_tables`, so agent traffic can be told apart from REST's. Saved-query and ad-hoc runs are recorded in run history (and live events) with `"transport": "mcp"`.

Limits are shared through Redis. With `QUERYAPIGATE_REDIS_URL` set (the same value for both processes), a key's `rate_limit` of 100/minute is 100 calls a minute over REST and MCP together. Without it, each process counts on its own: 100 over REST *and* 100 over MCP, as two instances would. Metrics are per process either way: `queryapigate mcp` serves its own `GET /metrics` (and `GET /health`) on its port, beside `/mcp` - open, or with `QUERYAPIGATE_METRICS_TOKEN` set, to a scraper sending it as a bearer token, as the REST server's. Scrape both.

The CLI is a local operator, not a caller. `queryapigate export` reads the store directly with no key, so no grant or rate limit applies - whoever can run it can already read `QUERYAPIGATE_HOME`. Its runs are still recorded in run history, with `key_name` `cli`.

Result size

A tool call caps its result at `QUERYAPIGATE_MCP_MAX_ROWS` (default 200) rows, independent of `QUERYAPIGATE_MAX_PAGE_SIZE` - an LLM's context window can't hold a result the way a human paging through the admin UI can. A capped result says so: `truncated` is `true`, and it carries an agent-facing `note` ("…more rows match; add a filter, or ask for a summary with an aggregate") both as a field in `structuredContent` and as a second text block - so a model narrows the query or aggregates instead of assuming the rows it got are the whole answer. The first text block stays pure JSON. A caller-supplied `page_size` smaller than the cap is still honored.

Configuration

VARIABLE	DEFAULT	EFFECT
`QUERYAPIGATE_MCP_PORT`	`5001`	Bind port for `queryapigate mcp`.
`QUERYAPIGATE_MCP_MAX_ROWS`	`200`	Row cap for a tool call's result. Always a positive count; a malformed value stops startup.
`QUERYAPIGATE_EMBEDDING_URL`	*(off)*	An OpenAI-compatible embeddings endpoint, to enable semantic search (`embed` parameters). Experimental; `https://` or `localhost`.
`QUERYAPIGATE_EMBEDDING_MODEL`	—	The embedding model name sent to that endpoint. Required when the URL is set.
`QUERYAPIGATE_EMBEDDING_API_KEY`	—	Bearer token for the embeddings endpoint, if it needs one. Omit for a local Ollama.

The OAuth settings are under Sign-in for AI clients.

These show up as a read-only "MCP server" section on the Console's Settings screen (BACKLOG #54) - the effective value and whether it's the default, same as every other setting there. The Console also has a dedicated **MCP** screen (in the sidebar) for the operational view: **Check now** tries a TCP connection to that port on `127.0.0.1` (only when asked - `GET /api/v1/mcp/status`), **Tools** lists what `tools/list` returns in the current workspace (`GET /api/v1/mcp/tools`), and a connect snippet shows the URL and header a client needs. Both are worked out by the REST server: the check only says whether *something* listens on that port on the same host, and the settings shown are the REST server's - an MCP process started with different variables isn't reflected. See Check the MCP server from the Console.